TrustHome PH

Firebase, Cloudinary & Security Setup

How the Firebase project, authentication, Firestore rules, Cloudinary uploads and credentials are configured today, and how to harden them.

Document	TH-04 · Firebase, Cloudinary & Security Setup
Version / date	Rev. 2.0 · 4 October 2026
Factual baseline	TrustHome_Current_Progress_Report_2026-10-04.md (PR #42 `fixes` working copy, based on `features/v1-3-navigation-settings`; excludes `dev`)
Replaces	Firebase Setup Plan
Audience	Team Idle developers, Scrum Master, QA, panel/adviser reviewers, and AI coding assistants used by the team

Evidence label	Meaning (never use these interchangeably)
[Implemented in source]	Code exists in the repository working copy. Not proof that it runs in production.
[Unit/emulator tested]	Covered by automated unit tests or the Firestore emulator suite.
[Reported working]	A team member observed it working in the browser against the test project.
[Deployed/live verified]	Deployed to trusthome-ph and confirmed working there.
[Known gap]	Does not yet meet the product claim. Do not present it as working.
[Roadmap] / **[Proposed]**	Planned future work / a recommendation in this document, not yet built.

Contents

1. Environment Facts . . . 3
2. One-Time Firebase Setup (for a new project or a rebuild) . . . 3
3. Firestore Security Rules . . . 4
3.1 Principles used in `firestore.rules` . . . 4
3.2 Illustrative patterns (the authoritative rules are in the repository) . . . 4
3.3 Testing and deploying rules . . . 5
4. Authentication Hardening Checklist . . . 5
5. Cloudinary Setup (current) . . . 5
6. Secure Verification-Document Delivery (P0 plan) [[prop]] . . . 6
6.1 Immediate actions . . . 6
6.2 Target design . . . 6
7. Secrets and Credential Hygiene . . . 6
8. Cloud Functions and the Blaze Decision . . . 7
9. Security Review Checklist (before any demo or release) . . . 7

1. Environment Facts

Item	Detail
Firebase project	`trusthome-ph` (configured in `src/firebase.js`). A teammate's local clone points at this **shared** project — there is no isolated per-developer project.
Billing plan	Spark (free). Cloud Functions and Firebase Storage are **not** available without Blaze.
Firestore location	asia-southeast1 (Singapore) recommended for the Philippines — confirm in console; location cannot be changed later.
Auth providers	Email/Password and Google
Rules / indexes	`firestore.rules`, `firestore.indexes.json` — deployed with the Firebase CLI. Rules deploy reported successful on 2026-10-04.
Hosting	Firebase Hosting, build folder `dist/`; last documented deploy 2026-10-01
File storage	Cloudinary, unsigned upload preset `trusthome_uploads`
Trusted credentials	Firebase Admin service account via `GOOGLE_APPLICATION_CREDENTIALS` or `FIREBASE_SERVICE_ACCOUNT_JSON`; GitHub secret `FIREBASE_SERVICE_ACCOUNT`

CAUTION · Shared project = shared data

- Every `npm run dev` writes to the same live database. Use clearly named test accounts (e.g. `qa.renter1@…`), prefix test listing titles with "[TEST]", and announce bulk changes in the team chat.
- For destructive experiments use the **Firestore emulator** (TH-05 §4), never the shared project.
- **[Proposed]** Create a second project (`trusthome-ph-dev`) and select it with a Vite env variable, so the demo project stays clean.

2. One-Time Firebase Setup (for a new project or a rebuild)

1. console.firebase.google.com → **Add project** → name `trusthome-ph` (or `trusthome-ph-dev`). Analytics not required.
2. **Project settings → Users and permissions**: add teammates. Give **Editor** only to those who deploy; **Viewer** to the rest. Keep exactly one or two Owners.
3. **Build → Firestore Database → Create database** in **production mode** (deny all) — not test mode — then deploy the repository rules immediately.
4. **Build → Authentication → Sign-in method**: enable Email/Password and Google. Under **Settings → Authorized domains** keep only `localhost`, the Hosting domains, and any custom domain.
5. **Project settings → Your apps → Web (</>)**: register `trusthome-web`, copy the config into `src/firebase.js`.
6. Install the CLI: `npm install -g firebase-tools`, `firebase login`, `firebase use trusthome-ph`.
7. Deploy rules and indexes: `firebase deploy --only firestore:rules,firestore:indexes --project trusthome-ph`.
8. Create the first admin: sign in once, then set `role: "admin"` on that `users/{uid}` document from the console or with the Admin SDK.

NOTE · The web config is not a secret

The `firebaseConfig` object (apiKey, projectId, …) identifies the project to Google. Security comes from Firestore rules and Auth settings. It may live in source. Service-account keys, the Cloudinary API secret and GitHub tokens are secrets and must never be committed.

3. Firestore Security Rules

3.1 Principles used in `firestore.rules`

- **Deny by default.** Anything not explicitly matched is denied.
- **Helper functions** for repeated checks: signed in, active profile, is admin, is owner.
- **Validate shape**: `request.resource.data.keys().hasOnly([...])` and `hasAll([...])`; type checks (`is string`, `is number`, `is timestamp`).
- **Validate transitions**: compare `resource.data.status` with `request.resource.data.status` and limit `diff(resource.data).affectedKeys()`.
- **Immutable ownership**: `ownerId`, `renterId`, `listingId` cannot change after create.
- **Server time**: require `request.time` for timestamps (`createdAt == request.time`).
- **Query safety**: list queries must include the filters the rules check (e.g. `ownerId == uid`).
- **Private split**: `listingPrivate` readable only by the active owner and admins.

3.2 Illustrative patterns (the authoritative rules are in the repository)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    function signedIn()   { return request.auth != null; }
    function me()         { return get(/databases/$(db)/documents/users/$(request.auth.uid)).data; }
    function active()     { return signedIn() && me().status == 'active'; }
    function isAdmin()    { return active() && me().role == 'admin'; }
    function changed()    { return request.resource.data.diff(resource.data).affectedKeys(); }

    match /users/{uid} {
      allow read:   if request.auth.uid == uid || isAdmin();
      allow update: if request.auth.uid == uid && !changed().hasAny(['role', 'status'])
                    || isAdmin();
    }

    match /listingPrivate/{id} {
      allow read: if active() && (resource.data.ownerId == request.auth.uid) || isAdmin();
    }

    match /bookings/{id} {
      // owner: Pending -> Confirmed only, touching only status/timestamps
      allow update: if active()
        && resource.data.ownerId == request.auth.uid
        && resource.data.status == 'Pending'
        && request.resource.data.status == 'Confirmed'
        && changed().hasOnly(['status', 'confirmedAt', 'updatedAt']);
    }

    match /trustScores/{id}    { allow read: if true; allow write: if false; }   // Admin SDK only
    match /{path=**}           { allow read, write: if false; }                  // deny-all fallback
  }
}
```

CAUTION · Rule cost and limits

Each `get()` in a rule is a billed read and there is a limit of 10 document-access calls per single-document request (20 for batches/transactions). Cache the profile in a function and avoid chains of lookups.

3.3 Testing and deploying rules

```
# run the focused emulator suite (needs Firebase CLI + a free emulator port; Java required by the emulator)
firebase emulators:exec --only firestore -- "node --test scripts/firestore-rules-smoke.mjs"

# deploy rules (and indexes) to the shared project
firebase deploy --only firestore:rules --project trusthome-ph
firebase deploy --only firestore:indexes --project trusthome-ph
```

BEST PRACTICE · Rules workflow

- Every rules change ships with at least one allow test and one deny test in `scripts/firestore-rules-smoke.mjs` (the suite had 9 passing tests in the latest run — grow it toward the full permission matrix in TH-08).
- Deploy rules **before** the frontend that depends on them, and only from the reviewed branch.
- Editing `firestore.rules` locally changes nothing live — record each deployment (date, commit, who) in the release log.
- Never "fix" a permission error by loosening a rule to `if true`; find which condition the request fails.

4. Authentication Hardening Checklist

- Authorized domains limited to localhost and real Hosting domains.
- Email enumeration protection enabled (Authentication → Settings → User actions) where available.
- Password policy: minimum length 8+ (raise from Firebase's default 6) and show strength feedback.
- Google sign-in and email/password with the same address are linked to one UID; profile creation is idempotent.
- **[Proposed]** Enable **Firebase App Check** (reCAPTCHA Enterprise / v3) for Firestore to reduce scripted abuse from outside the app.
- Suspension is enforced in rules through `status`, not only by hiding UI.

5. Cloudinary Setup (current)

1. Cloudinary console → note the **cloud name**.
2. Settings → Upload → Upload presets → `trusthome_uploads`, Signing mode **Unsigned**. Restrict allowed formats (jpg, jpeg, png, webp, pdf), set a max file size, and set a folder prefix (e.g. `trusthome/`).
3. In the app, `src/uploadImage.js` posts the file to the upload API. PDFs use resource type `raw`; images use `image`/`auto`. Listing-specific folders and tags organise assets.
4. Add teammates as Cloudinary users with the minimum role they need; never share the API secret in chat.

KNOWN GAP · What the unsigned preset does not do

- Anyone who obtains the preset name can upload to the account (abuse/quota risk). Restricting formats and size limits reduces this.
- Returned `secure_url` values are **public delivery URLs**. Storing them in `listingPrivate` hides them from other users of the app, but the file is not private.
- This account currently returns `show_original_customer_untrusted` for the uploaded PDF, so admins cannot preview or download it.

6. Secure Verification-Document Delivery (P0 plan) [Proposed]

Goal: identity and ownership documents are viewable only by the owner and admins, through links that expire within minutes, without ever making them public.

6.1 Immediate actions

1. Open a Cloudinary support ticket quoting `show_original_customer_untrusted`, explain the use case (admin review of user-submitted PDFs), and request account review and their secure-delivery guidance.
2. Check Cloudinary **Settings → Security** for the PDF/ZIP delivery restriction on the account and record what it shows; do not toggle public delivery of PDFs on as a workaround for identity documents.
3. Interim admin workaround only if needed for the demo: ask owners to upload ID/ownership evidence as images (JPG/PNG) — image delivery is not affected by the PDF block. Document that this is temporary.

6.2 Target design

Step	Mechanism
Upload	Signed upload with `type: authenticated` (or `private`) for documents. The signature is created by a trusted backend; the API secret never reaches the browser.
Store	`listingPrivate/{id}.documents` holds `public_id`, `resource_type`, `format`, `version` — not a URL.
View	Admin/owner requests a link; the backend checks Firebase ID token + role/ownership, then returns a signed delivery URL that expires in ~5 minutes.
Backend options	Firebase Callable Function (needs Blaze), or another serverless host the team controls that verifies Firebase ID tokens with the Admin SDK.
Migrate	Only after the app can read `documents` metadata: take and verify a backup → `migrate_verification_documents.py` dry run → review output → `--apply` → verify → remove old URL fields (the script does this only on success).

BEST PRACTICE · **Handling identity documents**

- Least exposure: never render documents on public pages, in notifications, logs, screenshots or these PDFs.
- Short-lived access beats hidden URLs; “unguessable” is not “private”.
- Define retention (e.g. delete or archive documents N days after a final decision) and honour owner deletion requests.
- Record admin access (who opened which document, when) once a backend exists.

7. Secrets and Credential Hygiene

Secret	Where it may live	Never
Firebase Admin service-account JSON	Local machine outside the repo (or ignored `bank_scraper/serviceAccountKey.json`), env var `GOOGLE_APPLICATION_CREDENTIALS` / `FIREBASE_SERVICE_ACCOUNT_JSON`, GitHub secret `FIREBASE_SERVICE_ACCOUNT`	Git, chat, PDFs, workbooks, screenshots, CI logs
Cloudinary API secret	Only in a trusted backend environment	Frontend code, Vite env vars (they are bundled into the client)
GitHub tokens	Personal credential manager	Shared docs

> **CAUTION · Current status**
> - A local `bank_scraper/serviceAccountKey.json` exists in the working copy and is ignored by both `.gitignore` files. It is not project source.
> - Whether any previously shared/committed key has been revoked is **not verified**. If a key was ever pasted in chat or committed: Google Cloud console → IAM → Service accounts → Keys → delete it, create a new one, update the GitHub secret, and check Git history (rewriting history does not un-leak a key — rotation does).

- Prefer a dedicated service account for jobs with only the Firestore roles it needs (e.g. Cloud Datastore User) instead of the default Admin SDK account with Editor rights.
- In GitHub Actions, write the secret to a temp file at runtime, never `echo` it, and delete it at the end of the job; GitHub masks registered secrets but not derived values.
- Run `git log -p -S "private_key"` periodically, or enable GitHub secret scanning, to catch accidental commits.

8. Cloud Functions and the Blaze Decision

`functions/` contains a callable booking confirmation (per-listing lock, copies the private address) and scheduled jobs. Deployment was blocked by the Blaze/Cloud Build requirement. Until deployed, the documented behaviour is the client transaction + Python jobs.

If the team enables Blaze	If the team stays on Spark
Set a **budget alert** (e.g. ₱100) in Google Cloud Billing immediately. Deploy functions: `firebase deploy --only functions`. Switch the UI to the callable for approval; keep the client transaction disabled. Use callable/HTTP functions for signed Cloudinary URLs. Move expiry/trust jobs to scheduled functions or keep them in Actions — not both.	Keep the client transaction and harden it with option B in TH-03 §4.6. Keep Python jobs in GitHub Actions (weekly) / trusted worker. Use a separate serverless host for document signing, or accept the documented limitation for the demo. Remove or clearly mark undeployed function code so nobody assumes it runs.

9. Security Review Checklist (before any demo or release)

- Rules deployed from the release commit; emulator suite green; deployment recorded.
- A signed-out browser cannot read pending listings, listingPrivate, bookings, disputes, users or notifications.
- A regular user cannot: approve own listing, change own role/status, confirm someone else's booking, write trustScores or bankProperties, read the `__admins__` feed.
- No document URL appears in any public listing document or page source.
- No secrets in the repository, built `dist/` bundle, or Actions logs.
- Admin accounts reviewed; departed testers removed from Firebase and Cloudinary.
- Known gaps (document delivery, booking race, address handoff) disclosed in demo notes.